Framework para Trabalho Prático de Programação Funcional

Projeto semestral em Haskell

Framework de proposta, desenvolvimento por fases, acompanhamento e avaliação

Projeto de referência: Sistema de Gestão de Alunos e Avaliações

1. Enquadramento e objetivo

O trabalho prático constitui um projeto semestral desenvolvido em grupo, em Haskell, que acompanha a progressão da unidade curricular de Programação Funcional. Cada grupo propõe um projeto próprio, sujeito a apresentação e aprovação do professor, e desenvolve-o de forma incremental ao longo do semestre.

O objetivo é que o projeto obrigue os estudantes a aplicar, num problema coerente e com dimensão suficiente, os principais fundamentos da programação funcional: modelação de dados, funções puras, recursão, funções de ordem superior, composição, tipos algébricos, tratamento explícito de erros e interação com o exterior através de IO.

```
modelação de dados  ->  núcleo funcional  ->  validação  ->  IO  ->  aplicação completa
```

2. Organização geral

Elemento	Definição
Constituição dos grupos	Grupos de estudantes definidos no início do semestre.
Linguagem	Haskell.
Tema	Livre, desde que adequado aos objetivos da UC e aprovado pelo professor.
Proposta	Obrigatória e apresentada pelo grupo antes do início do desenvolvimento formal.
Desenvolvimento	Três fases incrementais.
Fases 1 e 2	Entregas parcelares obrigatórias, apresentadas e validadas em aula.
Fase 3	Versão final do projeto, apresentada e defendida perante um júri.
Repositório	Utilização de Git desde o início do projeto.
Projeto de referência	Sistema de Gestão de Alunos e Avaliações.

3. Proposta inicial do projeto

Cada grupo deve apresentar uma proposta curta, suficientemente detalhada para permitir avaliar a adequação do projeto à UC. A proposta deve demonstrar que o problema escolhido tem dimensão para evoluir ao longo das três fases e que permite aplicar de forma natural os conceitos de programação funcional.

Elemento	Conteúdo esperado
Título	Nome curto e identificador do projeto.
Problema	Problema que a aplicação pretende resolver.
Objetivos	Resultados principais pretendidos.
Utilizadores	Quem utilizará o sistema e em que contexto.
Funcionalidades	Conjunto inicial de funcionalidades relevantes.
Dados	Principais entidades, estruturas e relações a modelar.
Entrada/Saída	Ficheiros, terminal, dados de entrada, relatórios ou outras formas de interação.

Componente funcional	Conceitos da UC que o projeto permitirá explorar.
Complexidade	Elementos que justificam a dimensão e o interesse do projeto.
Equipa	Identificação dos elementos do grupo.

3.1 Apresentação e aprovação

A proposta é apresentada ao professor em sessão curta. O objetivo desta apresentação é discutir o âmbito, identificar riscos e assegurar que o projeto é adequado à unidade curricular.

- Aprovada — o grupo pode avançar com o âmbito proposto.
- Aprovada com alterações — o grupo incorpora as alterações indicadas antes de iniciar o desenvolvimento.
- A reformular — a proposta não reúne ainda condições para avançar.

A aprovação da proposta funciona como um marco de arranque do projeto e não corresponde, por si só, a uma classificação autónoma.

4. Requisitos comuns a todos os projetos

Apesar de cada grupo escolher um domínio diferente, todos os projetos devem demonstrar um núcleo comum de competências. Os requisitos seguintes devem ser considerados mínimos e adaptados ao contexto concreto de cada projeto.

Área	**Requisito mínimo**
Modelação	Definição de tipos algébricos próprios (data), records e tipos adequados ao domínio.
Funções	Utilização predominante de funções puras no núcleo da aplicação.
Tipos	Assinaturas explícitas nas funções principais.
Recursão	Utilização justificada de funções recursivas.
Funções de ordem superior	Utilização de funções que recebem ou devolvem funções sempre que apropriado.
Processamento de coleções	Utilização de map, filter, folds e composição funcional.
Erros e ausência	Utilização adequada de Maybe e/ou Either.
IO	Interação com o utilizador, ficheiros ou outras fontes externas.
Persistência	Leitura e escrita de pelo menos um formato simples, quando aplicável.
Modularização	Código organizado em módulos com responsabilidades claras.
Testes	Testes das principais funções puras e dos casos limite.
Documentação	README com descrição, compilação, execução e estrutura do projeto.

5. Desenvolvimento em três fases

O desenvolvimento é incremental. Cada fase deve produzir uma versão coerente e demonstrável do trabalho realizado até esse momento. As Fases 1 e 2 são momentos de acompanhamento e validação; a Fase 3 corresponde à versão final e à avaliação perante o júri.

5.1 Fase 1 — Modelação e núcleo funcional

A primeira fase estabelece os fundamentos do projeto. O grupo deve definir o modelo de domínio, a estrutura inicial do código e as primeiras operações funcionais, privilegiando tipos, pattern matching e recursão.

```
tipos -> listas/estruturas -> pattern matching -> recursão -> núcleo do domínio
```

Elemento	Esperado na Fase 1
Modelo de domínio	Principais data types, records e relações entre entidades.
Estrutura do projeto	Módulos iniciais e organização do repositório.
Funções essenciais	Primeiras funções puras que implementam regras do domínio.
Recursão	Problemas adequados resolvidos recursivamente.
Dados de teste	Conjunto pequeno de dados para demonstração.
README	Descrição inicial e instruções básicas.
Git	Histórico de desenvolvimento do grupo.

A Fase 1 é apresentada em aula e classificada apenas como Validada ou Não validada.

5.2 Fase 2 — Lógica funcional e transformação de dados

Na segunda fase, o projeto deve evoluir para uma solução claramente funcional. Devem surgir funções de ordem superior, pipelines de transformação, composição, folds e tratamento explícito de resultados opcionais ou inválidos.

```
HOF -> map/filter -> fold -> composição -> Maybe/Either -> lógica da aplicação
```

Elemento	Esperado na Fase 2
Funcionalidades	Maioria da lógica de domínio implementada.
HOF	Utilização adequada de funções de ordem superior.
Pipelines	Uso de map, filter, folds e composição quando apropriado.
Erros	Tratamento explícito com Maybe e/ou Either.
Testes	Casos normais, casos limite e situações de erro.
Arquitetura	Separação crescente entre lógica funcional e IO.
Documentação	README e documentação atualizados.

A Fase 2 é apresentada em aula e classificada apenas como Validada ou Não validada.

5.3 Fase 3 — Aplicação final, apresentação e defesa

A terceira fase corresponde ao projeto final. O grupo deve apresentar uma aplicação integrada, utilizável e suficientemente robusta para demonstrar o funcionamento de ponta a ponta. Esta fase inclui a

apresentação e defesa perante um júri e é a única fase à qual é atribuída uma classificação numérica de 0 a 20 valores.

```
entrada/ficheiros  ->  parsing/validação  ->  núcleo funcional  ->  saída/relatórios
```

Elemento	Esperado na Fase 3
Aplicação	Versão completa, executável e demonstrável.
IO	Entrada e saída claramente separadas do núcleo funcional.
Ficheiros/dados	Leitura e escrita devidamente integradas.
Parsing e validação	Tratamento seguro dos dados externos.
Erros	Tratamento consistente de situações anómalas.
Modularização	Estrutura final do código por módulos.
Testes	Conjunto de testes das funcionalidades principais.
Qualidade	Código legível, simples, sem duplicação desnecessária.
Documentação	README final e documentação de suporte à apresentação.

6. Estrutura recomendada da entrega final

```
projeto/
├── README.md
├── cabal.project
├── projeto.cabal
├── src/
│   ├── Model.hs
│   ├── Domain.hs
│   ├── Parsing.hs
│   └── ...
├── app/
│   └── Main.hs
├── test/
│   └── ...
├── data/
│   └── ...
└── docs/
    └── relatorio.pdf
```

O README deve permitir que uma terceira pessoa compreenda o objetivo do projeto, o compile e o execute sem explicações adicionais por parte do grupo.

7. Relatório final

O relatório deve ser sintético e centrado nas decisões de conceção e programação. Não se pretende um relatório académico extenso; pretende-se documentação técnica suficiente para justificar a solução implementada.

Secção	Conteúdo
1. Projeto	Problema, motivação e objetivos.

2. Requisitos	Principais funcionalidades.
3. Modelo	Tipos de dados fundamentais e decisões de modelação.
4. Arquitetura	Organização em módulos e separação de responsabilidades.
5. Programação funcional	Decisões funcionais relevantes e exemplos representativos.
6. Tratamento de erros	Utilização de Maybe, Either e validações.
7. Testes	Estratégia e exemplos de testes.
8. Evolução	Principais alterações introduzidas entre as fases.
9. Conclusões	Limitações e possíveis extensões.
10. Contribuições	Identificação do contributo de cada elemento do grupo.

8. Modelo de avaliação

A avaliação assenta numa lógica de acompanhamento obrigatório nas Fases 1 e 2 e de avaliação final na Fase 3. As duas primeiras fases não atribuem valores positivos à classificação final: são checkpoints que podem gerar penalização quando não são validados.

Fase	Momento	Resultado	Efeito na classificação
Fase 1	Apresentação em aula	Validada / Não validada	Não validada: -3 valores
Fase 2	Apresentação em aula	Validada / Não validada	Não validada: -3 valores
Fase 3	Apresentação e defesa perante júri	0 a 20 valores	Determina a nota base/final

Regra de cálculo: a nota final corresponde à classificação obtida na Fase 3, deduzida de 3 valores por cada uma das Fases 1 e 2 que não tenha sido validada.

```
Nota Final = Nota da Fase 3 - 3 x N
N = número de fases não validadas entre a Fase 1 e a Fase 2
```

Fase 1	Fase 2	Nota da Fase 3	Penalização	Nota final
Validada	Validada	16	0	16
Não validada	Validada	16	-3	13
Validada	Não validada	16	-3	13
Não validada	Não validada	16	-6	10

A não validação de uma fase não impede o grupo de continuar a desenvolver o projeto e de corrigir os problemas identificados. Contudo, a penalização associada à fase mantém-se na classificação final.

9. Apresentação e defesa perante o júri

A Fase 3 termina com uma apresentação e defesa perante um júri. O objetivo é avaliar simultaneamente a qualidade do produto final, a utilização dos princípios de programação funcional e o domínio individual demonstrado pelos elementos do grupo.

Momento	Conteúdo sugerido

Problema e objetivos	Enquadramento e principais requisitos.
Arquitetura e modelação	Tipos principais, módulos e decisões de conceção.
Demonstração	Execução das funcionalidades mais relevantes.
Programação funcional	Exemplos de código que evidenciem os conceitos da UC.
Defesa	Questões do júri sobre código, tipos, decisões, erros, testes e alternativas.

9.1 Critérios observáveis na Fase 3

- Correção e completude da aplicação.
- Adequação da modelação de dados.
- Qualidade e clareza do código Haskell.
- Utilização efetiva dos princípios de programação funcional.
- Uso adequado de tipos, pattern matching, recursão e funções de ordem superior.
- Uso de map, filter, folds e composição quando apropriado.
- Tratamento de ausência e erro com Maybe e/ou Either.
- Separação entre núcleo funcional e IO.
- Modularidade, testes e documentação.
- Qualidade da apresentação e domínio demonstrado na defesa.

10. Responsabilidade individual no trabalho de grupo

Embora o projeto seja desenvolvido em grupo, todos os elementos devem conhecer a solução global e ser capazes de explicar o código apresentado. O professor e o júri podem utilizar o histórico Git, as apresentações parcelares e as respostas individuais na defesa para avaliar a participação e o domínio de cada estudante.

Quando se justifique, a classificação individual poderá refletir diferenças claras de participação ou de domínio demonstrado, de acordo com as regras gerais de avaliação da unidade curricular.

11. Articulação indicativa com o semestre

Período	**Conteúdos da UC**	**Projeto**
Semanas 1-2	Introdução, funções e tipos	Formação dos grupos e discussão de ideias.
Semana 3	Listas, pattern matching e guards	Apresentação e aprovação da proposta.
Semanas 4-5	Recursão	Desenvolvimento e apresentação da Fase 1.
Semanas 6-9	HOF, folds, composição, ADTs e estruturas recursivas	Desenvolvimento e apresentação da Fase 2.
Semanas 10-13	Type Classes, Maybe/Either, IO e lazy evaluation	Desenvolvimento da aplicação integrada.
Final do semestre	Síntese da UC	Fase 3: apresentação e defesa perante júri.

12. Projeto de referência — Sistema de Gestão de Alunos e Avaliações

O Sistema de Gestão de Alunos e Avaliações pode ser utilizado pelo professor como exemplo transversal da framework. O projeto serve para ilustrar o nível de detalhe e a evolução esperada em cada fase, sem constituir um modelo obrigatório de tema para os grupos.

Fase	Exemplo de evolução no projeto de referência
Proposta	Gestão de alunos, avaliações, classificações, pesquisa e relatórios.
Fase 1	Tipos Aluno, UnidadeCurricular e Avaliacao; funções básicas de pesquisa, classificação e cálculo.
Fase 2	Validação, estatísticas, rankings, pipelines com map/filter/folds e tratamento com Maybe/Either.
Fase 3	Leitura de ficheiros, parsing, menu ou interface textual, geração de relatórios, testes e demonstração completa.

13. Princípios orientadores

- O objetivo não é maximizar a quantidade de código, mas produzir uma solução correta, simples, clara e funcional.
- A modelação dos dados deve anteceder a implementação das funcionalidades.
- As funções puras devem concentrar a lógica de domínio; o IO deve ficar limitado à interação com o exterior.
- As abstrações estudadas devem ser utilizadas quando simplificam a solução e tornam o código mais expressivo.
- Cada fase deve refletir evolução real do projeto e incorporar o feedback recebido anteriormente.
- O grupo deve conseguir justificar as principais decisões de tipos, funções, tratamento de erros, modularização e testes.

Uma solução pequena, clara, tipada e composicional é preferível a uma solução extensa que reproduza padrões imperativos.